Object-Oriented Programming — Week 3

Formatted by: Heisenberg 😎

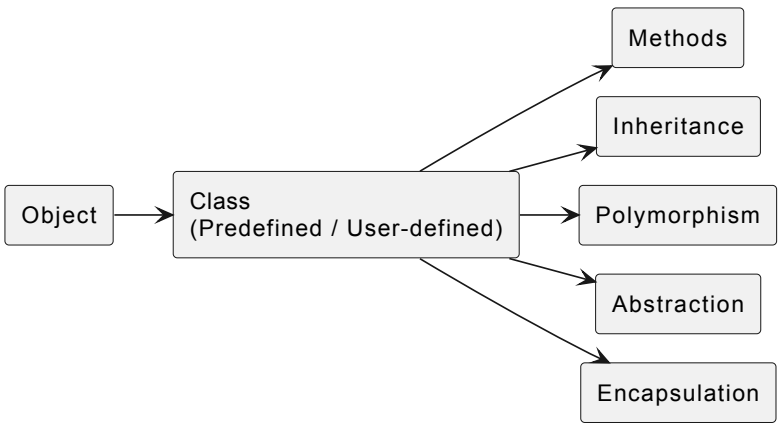
Index

- Where We Left Off — a quick "previously on OOP" recap so nobody feels lost.
- The Real-World Problem: Bicycle Tracking — because apparently every CS course needs a vehicle at some point.
- Defining and Using a Class — meet `Bicycle`, the class that owns nothing but an owner's name.
- The Sample Program: BicycleRegistration — two classes, one campus full of bikes.
- Program Output and Dependency Diagram — who's calling whom, and who gets to say "I own a bicycle."
- Defining Your Own Classes — the paperwork behind making a class official.
- The Bicycle Class in Detail — constructor, getter, setter: the holy trinity of a simple class.
- Data Member Declaration — naming your variables like a responsible adult.
- Accessibility (Visibility) Modifiers — `private` vs `public`, or "who's allowed in the clubhouse."
- Void Methods — methods that do the work and ask for nothing in return.
- Single Class vs Multiple Classes — the "main character" of your program.
- Constructors: Default vs Parameterized — initializing objects with either nothing or something.
- Practice Questions — five questions to make sure this actually stuck.

Where We Left Off

This session continues directly from the previous class on object-oriented programming. So far, the course has covered the **class hierarchy** and the broad features that sit inside it: **inheritance**, **overriding**, **polymorphism**, and **abstract classes**. On the conceptual side, the course has already introduced **objects** and **classes** — distinguishing between **predefined classes** (the ones Java ships with) and **user-defined classes** (the ones we write ourselves) — along with the core pillars of OOP: inheritance, polymorphism, abstraction, and encapsulation.

Until now, every example has used a **single class** containing its own data members and methods. This session's focus is the natural next step: what happens when a real program needs **multiple classes**, each with its own members and methods, working together.



> **Note for Review:** Every OOP topic from here on builds on three ingredients — **object**, **class**, and **method**. Whenever a new concept (inheritance, polymorphism, abstract classes) feels overwhelming, ground yourself by asking: "which of these three is changing here?"

The Real-World Problem: Bicycle Tracking

Good class design should map to a **real-life application** — a program is only as useful as the problem it solves. The example chosen here is deliberately close to home: on a residential campus (like IIT Jodhpur), the most economical and efficient means of on-campus transportation is, without doubt, a bicycle, and students use bicycles constantly to get around.

Problem statement: design a program that *tracks* bicycles by assigning each one some form of **identification number**, along with relevant information such as the **owner's name** and **phone number**.

To build this in Java, we need to design several different kinds of objects — not just one. A working application typically needs objects to handle:

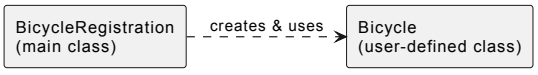
- **Input**
- **Output**
- **Data storage**
- Other **computational tasks** (e.g., counting how many bicycles are parked in a given stand)

Among all these necessary objects, this lecture focuses on designing **one class that models a bicycle itself**. There is no `Bicycle` class among Java's standard classes — it isn't predefined — so we must define it ourselves. A class we define ourselves (as opposed to one built into Java) is called a **programmer-defined class** (also referred to as a **user-defined class**).

The analogy used in class: registering a bicycle is conceptually the same as registering a car or a bike with an RTO (Regional Transport Office) — you get an identification number, and that number is linked to an owner's name. This is exactly the relationship the two classes in this example will model.

Defining and Using a Class

We'll start with a very simple `Bicycle` class. Using this first version, we can only **assign** and **retrieve** the owner's name — nothing more. Before looking inside the `Bicycle` class itself, it helps to see how it's *used* in a program, since that usage pattern is the same for every class you'll ever write.



For this sample program, two classes are created:

- **`BicycleRegistration`** — the **main class**, because it contains the `main` method.
- **`Bicycle`** — a supporting class that does **not** contain `main`.

Since Java conventionally uses one class definition per file, named after the class itself, this means two separate source files:

- `BicycleRegistration.java`
- `Bicycle.java`

The Sample Program: BicycleRegistration

```
class BicycleRegistration {
    public static void main(String[] args) {
        Bicycle bike1, bike2;
        String owner1, owner2;

        bike1 = new Bicycle();   // Create and assign values to bike1
        bike1.setOwnerName("Adam Smith");

        bike2 = new Bicycle();   // Create and assign values to bike2
        bike2.setOwnerName("Ben Jones");

        // Output the information
        owner1 = bike1.getOwnerName();
        owner2 = bike2.getOwnerName();

        System.out.println(owner1 + " owns a bicycle.");
        System.out.println(owner2 + " also owns a bicycle.");
    }
}
```

Walking through what happens here, framed as a registration process (just like registering a real bike or car and getting a number plate):

1. Two `Bicycle`-type variables (`bike1`, `bike2`) and two `String` variables (`owner1`, `owner2`) are declared.
2. `bike1 = new Bicycle();` **creates a new object** belonging to the `Bicycle` class — in OOP, before you can do anything with a class, you must instantiate an **object** from it.
3. `bike1.setOwnerName("Adam Smith");` assigns an owner's name to that specific bicycle object — this is the "registration" step, analogous to linking a person's name to a specific vehicle number.
4. The same two steps repeat for `bike2`, registered to "Ben Jones."
5. `getOwnerName()` is then used to **retrieve** each owner's name back out of the object and store it in a `String` variable, so it can be displayed.
6. Finally, `System.out.println` displays which person owns which bicycle.

This is the essential registration pattern: create an object → set its data → get its data back out → display it.

The `Bicycle` Class Definition

```
class Bicycle {
    // Data Member
    private String ownerName;

    // Constructor: Initializes the data member
    public Bicycle() {
        ownerName = "Unknown";
    }

    // Returns the name of this bicycle's owner
    public String getOwnerName() {
        return ownerName;
    }

    // Assigns the name of this bicycle's owner
    public void setOwnerName(String name) {
        ownerName = name;
    }
}
```

Here, `Bicycle()` — a special function sharing the exact same name as the class — is the **constructor**. This particular constructor takes **no arguments**, and its only job is to initialize the data member `ownerName` to `"Unknown"` whenever no name has been given yet.

Method	Parameter	Description
`Bicycle`	None	Initializes the owner's name to `Unknown`.
`getOwnerName`	None	Returns the owner's name.
`setOwnerName`	Name of the owner (string)	Assigns the bicycle owner's name to the passed value.

Table: The three methods of the `Bicycle` class. The first method listed is the constructor.

Program Output and Dependency Diagram

The relationship between the two classes can be visualized as a simple dependency: `BicycleRegistration` depends on `Bicycle` (it creates and uses `Bicycle` objects), but `Bicycle` does not need to know anything about `BicycleRegistration`.



Conceptually: every bicycle must first be "registered" through `BicycleRegistration` (the class holding `main`), and it is `BicycleRegistration` that calls into the `Bicycle` class to `set` and `get` values.

When the program runs, it produces the following output on the standard output window:

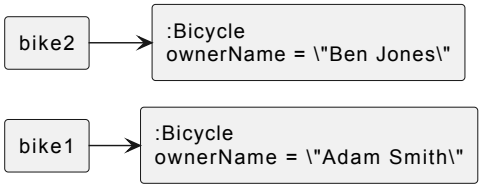
```
Adam Smith owns a bicycle.
Ben Jones also owns a bicycle.
```

Both output lines come from the same underlying pattern: a name was **set** via `setOwnerName()` (defined in the `Bicycle` class) and then **retrieved** via `getOwnerName()` — the method that returns the value stored in that object.

Multiple objects, one class: Note that two separate `Bicycle` objects (`bike1` and `bike2`) were created from the *same* `Bicycle` class — this is exactly the point of a class: it's a reusable blueprint, and you can create as many bicycles (objects) from it as your campus actually has.

Memory State After Object Creation

Each object created from a class gets its **own independent copy** of the class's instance variables. After both bicycles are created and assigned their names, the memory looks like this:



`bike1` and `bike2` are reference variables pointing to two **separate** `Bicycle` objects in memory, each carrying its own value of `ownerName`. This is exactly what makes `ownerName` an **instance variable**: it's a data member associated with an individual instance, and its value can change independently from object to object over time. (Class-level variables shared across *all* instances — as opposed to instance variables — will be covered later in the course.)

Defining Your Own Classes

Structurally, using a programmer-defined class like `Bicycle` looks exactly the same as using any standard Java class — that's the whole point of OOP: once defined, a class behaves like a first-class citizen of the language.

- Create an object: `bike2 = new Bicycle();`
- Call a method to store data: `bike2.setOwnerName("Ben Jones");`
- Call a method to retrieve data: `bike2.getOwnerName();`
- Store the returned value:

```
String owner2;
...
owner2 = bike2.getOwnerName();
```

By convention (not a strict requirement), Java saves **one class definition per file**, and the file is named after the class with a `.java` suffix — hence `Bicycle.java` for the `Bicycle` class.

General Template for a Class

```
class Bicycle {
    // data members

    // methods
}
```

The `Bicycle` Class in Detail

Before writing method definitions, we must decide the **data members** — the data values associated with the class or its instances (compare: the current balance of an `Account` object). For `Bicycle`, we only need to track one thing per object: the **owner's name**. Data members are declared inside the class declaration:

```
class Bicycle {
    private String ownerName;

    // definitions for the constructor,
    // getOwnerName, and setOwnerName methods come here
}
```

`ownerName` here is an **instance variable**: every object created from the class gets its own independent copy of it, and that value can change over the object's lifetime (as opposed to class constants or class variables, covered in a later chapter).

Data Member Declaration

The general syntax for declaring a data member is:

```
<modifier-list> <data type> <name>;
```

Component	Meaning
`<modifier-list>`	Designates different characteristics of the data member (e.g., accessibility)
`<data type>`	A class name or a primitive data type
`<name>`	The name of the data member

Applied to our example:

`private` `String` `ownerName ;`

```
private String ownerName;
```

Here, `private` is the **modifier**, `String` is the **data type**, and `ownerName` is the **name**.

Accessibility (Visibility) Modifiers

The modifier `private` used above is an example of an **accessibility modifier** (also called a **visibility modifier**). It restricts *who* can directly access a data member.

- **`private`** — only the methods **defined inside that same class** can access the member directly. No outside class or program can touch it.
- **`public`** — accessible from **anywhere**, including programs and classes outside the one that defines it.

As a rule of thumb followed throughout this course: **data members are almost always declared `private`**, to keep an object's internal state protected from direct outside interference — this is the essence of **encapsulation**.

Recall the `setOwnerName` method:

```
public void setOwnerName(String name) {
    ownerName = name;
}
```

The general syntax for defining *any* method is:

```
<modifiers> <return type> <method name> ( <parameters> ) {
    <statements>
}
```

Mapped onto `setOwnerName`:

Syntax component	Value in this method
Modifier	`public`
Return type	`void`
Method name	`setOwnerName`
Parameter	`String name`
Statement	`ownerName = name;`

Inside the body, `ownerName` refers to the **instance variable**, while `name` refers to the **parameter** passed in. Keeping this distinction clear — instance variable vs. parameter — is exactly why the naming convention (different names, or use of `this.ownerName`) matters.

> **Pro-Tip:** This distinction between "the class's own variable" and "the parameter with a similar name" is a classic exam trap. When you see `ownerName = name;` inside `setOwnerName(String name)`, remember: the left-hand side is the object's permanent state; the right-hand side is a temporary value handed in by whoever called the method.

Void Methods

A method may or may not return a value. A method that does **not** return anything — like `setOwnerName` — is declared with the return type `void`, and is called a **void method**. Its entire job is to *do* something (here, update the internal state), not to hand back a result.

```
public void setOwnerName(String name) {
    ownerName = name;
}
```

The accessibility modifier for `setOwnerName` is `public`, meaning any program using the `Bicycle` class can call it directly. Methods *can* also be declared `private` — a private method cannot be called from outside the class at all; it can only be called by **other methods within the same class**. This lecture deliberately limits itself to `public` methods for now, since we specifically want external programs (like `BicycleRegistration`) to be able to call them; private methods will be covered later in the course.

Contrast: A Method That *Does* Return a Value

```
public String getOwnerName() {
    return ownerName;
}
```

Syntax component	Value in this method
Modifier	`public`
Return type	`String`
Method name	`getOwnerName`
Parameters	None
Statement	`return ownerName;`

Here, `ownerName` inside the method body again refers to the instance variable, and the method sends that value back out to whoever called it (`return ownerName;`) — the opposite job of a void method.

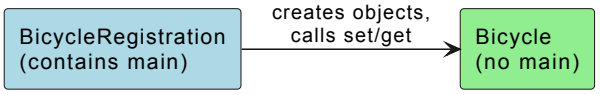
Single Class vs Multiple Classes

Up to this point in the course, only **single-class** programs had been written. This lecture's core shift is to **multiple classes**, where:

- A program may be split across several class files, but
- **Only one class contains the `main` method** — this is the class from which the program actually starts executing, and it is informally called the **main class**.
- Every other class does *not* contain `main`; it exists to be **used** by the main class (or by other classes) — objects and values are created from it, retrieved from it, and passed around, but execution doesn't *start* there.

In this example:

- `BicycleRegistration` → contains `main` → **the main class**.
- `Bicycle` → does not contain `main` → a supporting, user-defined class used by the main class.



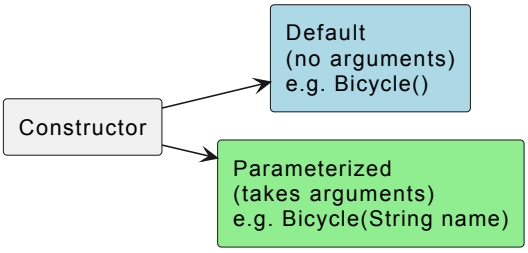
Every object-oriented program, regardless of size, is ultimately built from the same three ingredients: **objects**, **classes**, and **methods**.

Constructors: Default vs Parameterized

For initializing an object's data when it's first created, Java uses **constructors** — a special method sharing the class's exact name, with no return type at all (not even `void`).

Two flavors were introduced:

- **Default constructor** — takes **no arguments**. Used when there's no specific starting value to assign, so the data member is initialized to a placeholder like `"Unknown"`. This is exactly what `Bicycle()` does in this example.
- **Parameterized constructor** — takes one or more **arguments**, and uses those passed-in values to initialize the data member(s) directly at creation time, instead of relying on a separate `set` call afterward.



This distinction — default vs. parameterized construction — becomes especially relevant later when the course covers **method overriding** and **dynamic method dispatch**, both of which rely on a solid understanding of how objects are constructed and initialized across multiple, related classes.

Practice Questions

1. In the `Bicycle` class, `ownerName` is declared `private`, but `getOwnerName()` and `setOwnerName()` are declared `public`. If you changed `ownerName` itself to `public` and deleted both the getter and setter, would the program in `BicycleRegistration` still behave identically? What would you lose by making that change, even if the output stayed the same?

2. Suppose `BicycleRegistration` tried to call `bike1.ownerName = "Adam Smith";` directly instead of using `setOwnerName("Adam Smith")`. Under the current class design, would this compile? Explain *why* the accessibility modifier is the deciding factor here, not the fact that a value is simply being assigned.

3. The default constructor `Bicycle()` initializes `ownerName` to `"Unknown"`. Design (in words or code) a **parameterized constructor** `Bicycle(String name)` that initializes `ownerName` directly from an argument. Then explain: once you add this parameterized constructor, does the original no-argument `new Bicycle()` call in `BicycleRegistration` still work without any other change? Why or why not?

4. `BicycleRegistration` is the "main class" because it contains `main`, while `Bicycle` is not. If a school wanted to track not just bicycles but also *scooters* and *skateboards* on campus, would you add a `main` method to each new vehicle class, or keep a single main class? Justify your answer in terms of how many entry points a Java program is allowed to actually use when it runs.

5. Two objects, `bike1` and `bike2`, are created from the same `Bicycle` class and each stores a different `ownerName`. If you wrote `bike2 = bike1;` right after both objects were set up (instead of creating `bike2` with `new Bicycle()`), what would happen to `bike2`'s original owner data, and how many *distinct* `Bicycle` objects would actually exist in memory afterward? Use the earlier memory-state diagram of `bike1` / `bike2` as reference for your reasoning.

References

1. *A Comprehensive Introduction to Object-Oriented Programming With Java* by C. Thomas Wu — ISBN-13: 978-0071285858
2. *Java: The Complete Reference*, 13th Edition, by Herbert Schildt — ISBN-13: 978-9355326478